

HW3.8

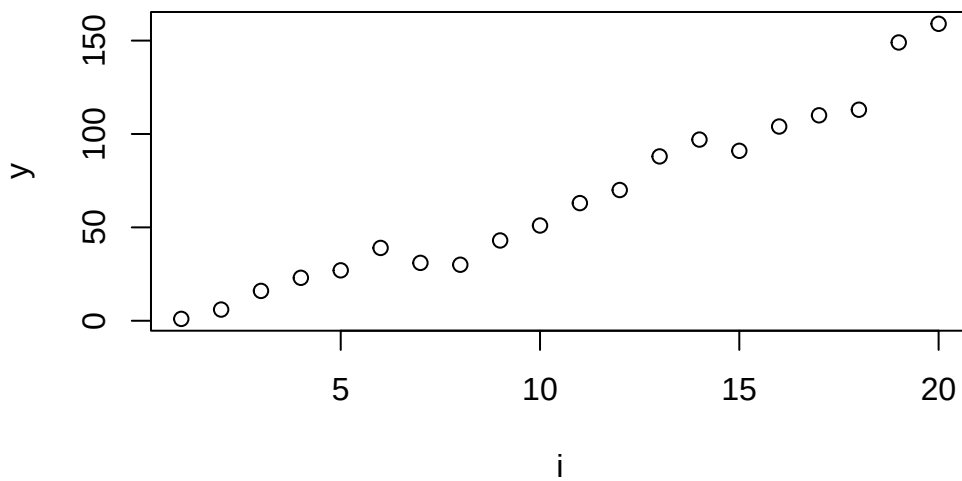
RJ Cass

1

```
y <- c(1, 6, 16, 23, 27, 39, 31, 30, 43, 51, 63, 70, 88, 97, 91, 104, 110, 113, 149, 159)
i <- 1:20
```

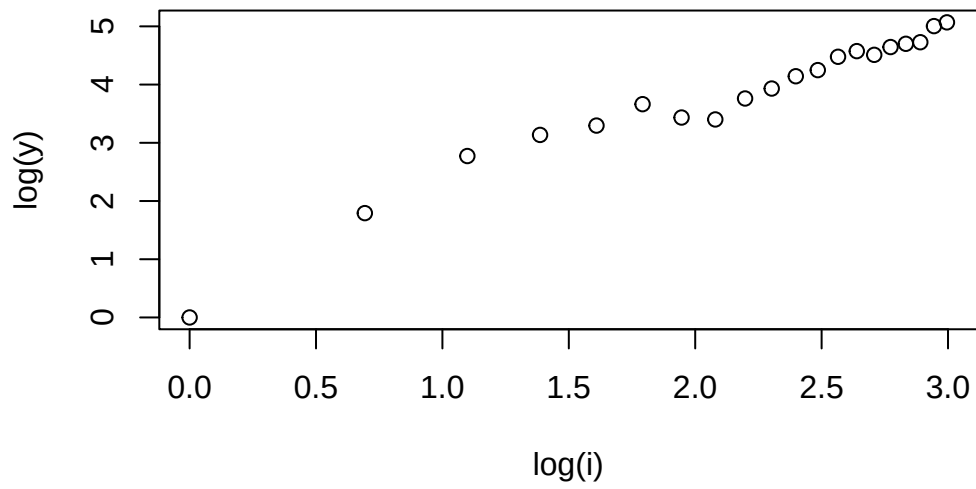
(a) Make a plot of the number of cases per time period.

```
plot(i, y)
```



(b) Plot the log of the response and covariate

```
plot(log(i), log(y))
```



(c) Fit a GLM using first principles

$$y \sim \text{Pois}(\mu_i), \log(\mu_i) = \eta_i, \eta_i = x\beta$$

$$\mu_i = e^{\eta_i} = e^{x\beta}$$

Due to being Poisson, $\text{Var}(y_i) = \mu_i$

$$g'(\mu_i) = \frac{1}{\mu_i}$$

```
X <- cbind(rep(1, length(i)), log(i)) #X matrix
b0 <- 0
b1 <- 0
b <- c(b0, b1)
epsilon <- .00001
diff <- 1
iter <- 0

while(diff > epsilon) {
  # Poisson Exponential Link
  mui <- exp(X%*%b)
  # Poisson variance = mean
  varyi <- mui
  # Derivative of the link function of mui
  gprimemui <- 1/mui
```

```

z <- X%*%b + (y-mui)*gprimemui
W <- diag(c(1/(varyi*gprimemui^2)))
b <- solve(t(X)%*%W%*%X)%*%t(X)%*%W%*%z
diff <- (b[1]-b0)^2 + (b[2]-b1)^2
b0 <- b[1]
b1 <- b[2]
iter <- iter + 1
# print(paste("Iteration: ", iter))
# print(paste("Estimate: ", b))
}

print(paste("Estimate (int): ", round(b[1], 4), "Estimate (log(i)): ", round(b[2], 4)))

```

```
[1] "Estimate (int): 0.996 Estimate (log(i)): 1.3266"
```

(d) Fit GLM using R packages

```

mod <- glm(y ~ log(i), family = poisson)
round(summary(mod)$coefficients[,1], 4)

```

```

(Intercept)      log(i)
      0.9960      1.3266

```

I got basically the exact same results from both methods.

(e) Find the covariance of β

```

# Get the covariance using the final value of b
mui <- exp(X%*%b)
varyi <- mui
gprimemui <- 1/mui
W <- diag(c(1/(varyi*gprimemui^2)))
J <- t(X)%*%W%*%X
cov <- solve(J)
cov

```

```

      [,1]      [,2]
[1,] 0.02880067 -0.01082261
[2,] -0.01082261 0.00417752

```

```

# Confidence interval for Beta0
b0_int <- round(b0 + c(-1, 1)*(1.96*sqrt(cov[1,1])), 4)

# Confidence interval for Beta1
b1_int <- round(b1 + c(-1, 1)*(1.96*sqrt(cov[2,2])), 4)

print(paste("Beta_0 lower: ", b0_int[1], "Beta_0 upper: ", b0_int[2]))

```

```
[1] "Beta_0 lower: 0.6634 Beta_0 upper: 1.3286"
```

```
print(paste("Beta_1 lower: ", b1_int[1], "Beta_1 upper: ", b1_int[2]))
```

```
[1] "Beta_1 lower: 1.1999 Beta_1 upper: 1.4533"
```

Yes, I found that $\log(i)$ is significant in the model because the confidence interval on the coefficient estimate does not include 0.

(f) Plot the 4 default plots using first principles

I used chat quite a bit to figure out how to piece together all the math, and especially how to get those ridiculous Quantile thingies.

```

# Some stuff to calculate to use later in the graphs

# Theoretical Half Quantiles
N <- length(i)
ranks <- 1:N
a <- if (N <= 10) 0.375 else 0.5
probabilities <- (ranks - a) / (N + 1 - 2 * a)
# Instead of mapping 'probabilities' directly to qnorm(),
# we shift them to the upper half: (probabilities + 1) / 2
half_normal_probabilities <- (probabilities + 1) / 2
theoretical_half_quantiles <- qnorm(half_normal_probabilities)

# Hat Matrix

```

```

H <- sqrt(W) %*% X %*% solve(t(X) %*% W %*% X) %*% t(X) %*% sqrt(W)
# Leverage
leverage <- diag(H)

# Pearson Residuals
mui <- c(mui)
p_resids <- (y - mui)/sqrt(mui)
# Standardized Pearson Residuals
std_p_resids <- (y - mui)/sqrt(mui*(1-leverage))

# Deviance Resids
dev_resid <- 2 * (ifelse(y == 0, 0, y * log(y / mui)) - (y - mui))
dev_resid <- sign(y - mui) * sqrt(dev_resid)
abs_dev_resids <- abs(dev_resid)
sorted_resids <- sort(abs_dev_resids)
abs_std_dev_resids <- abs_dev_resids / sqrt(1 - leverage)

# Slope and Intercept for QQ plot line
y_coords <- quantile(abs_std_dev_resids, probs = c(0.25, 0.75), type = 7)
x_coords <- qnorm((c(0.25, 0.75) + 1) / 2)
slope <- diff(y_coords) / diff(x_coords)
intercept <- y_coords[1] - slope * x_coords[1]

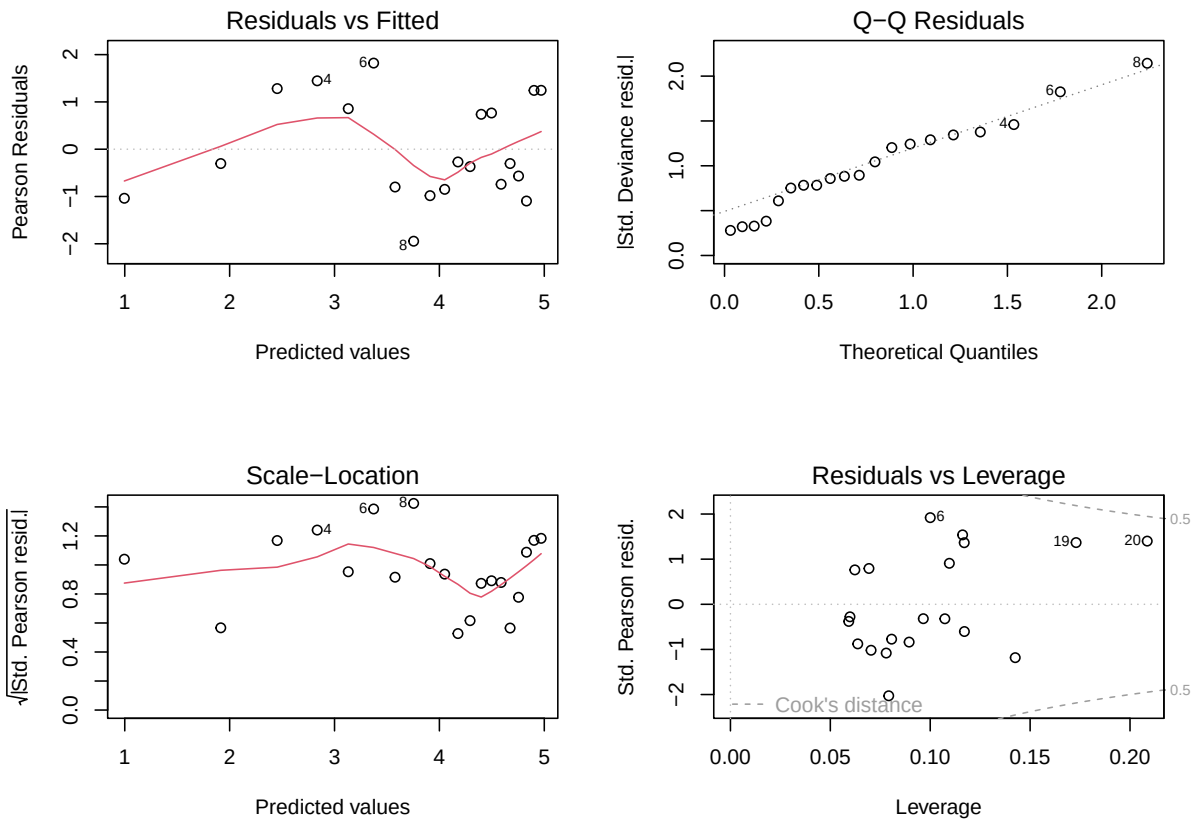
```

Here are the plots actually generated via plot(mod)

```

# Actual plots
par(mfrow = c(2, 2))
plot(mod)

```

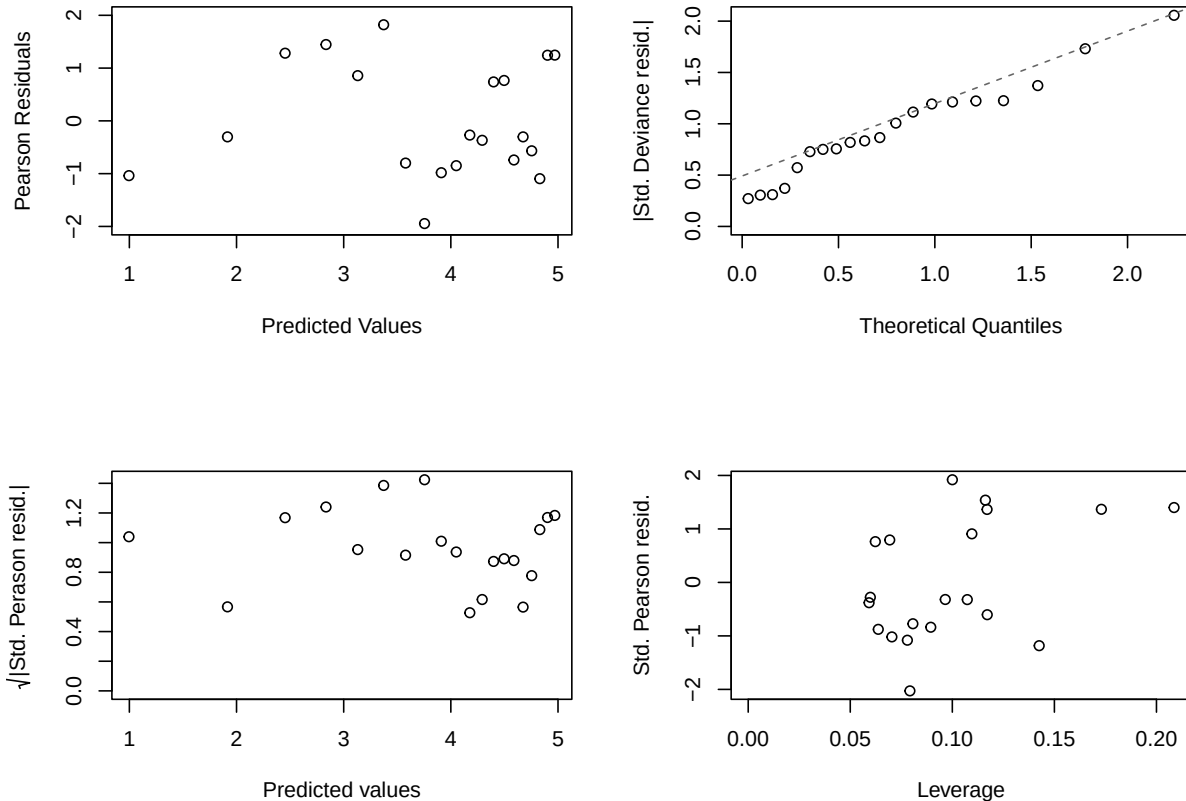


```
par(mfrow = c(1, 1))
```

Here are my recreations of those plots. I couldn't quite get the line on the QQ plot to match the same intercept/slope for some reason :(

```
par(mfrow = c(2, 2))
# Plot 1
plot(log(mui), p_resids, ylim = c(-2, 2),
     xlab = "Predicted Values",
     ylab = "Pearson Residuals")
# Plot 2
plot(theoretical_half_quantiles, sorted_resids, ylim = c(0, max(sorted_resids)),
     xlab = "Theoretical Quantiles",
     ylab = "|Std. Deviance resid.|")
abline(a = intercept, b = slope, lty = 2, col = "gray40")
# Plot 3
plot(log(mui), sqrt(abs(std_p_resids)), ylim = c(0, max(sqrt(abs(std_p_resids)))),
     xlab = "Predicted values",
     ylab = expression(sqrt("|Std. Pearson resid.|"))
```

```
# Plot 4
plot(leverage, std_p_resids, xlim = c(0, max(leverage)),
     xlab = "Leverage",
     ylab = "Std. Pearson resid.")
```



```
par(mfrow = c(1, 1))
```

(g) Find 95% interval on prediction of nect time period

```
# Manual Method
# New data point
new_data <- data.frame(i = 21)
X_new <- model.matrix(~ log(i), data = new_data)
# Predicted eta
eta_hat <- sum(X_new %*% b)
# Error of eta
se_eta <- sqrt(X_new %*% cov %*% t(X_new))
```

```

z_crit <- qnorm(0.975)
ci_eta_lower <- eta_hat - z_crit * se_eta
ci_eta_upper <- eta_hat + z_crit * se_eta
# Transform to counts
first_principle_pred <- data.frame(
  fit = exp(eta_hat),
  lwr = exp(ci_eta_lower),
  upr = exp(ci_eta_upper)
)

# Using R's predict method
glm_pred <- predict(mod, newdata = new_data, type = "link", se.fit = TRUE)
z_val <- qnorm(0.975) # 1.96
log_lwr <- glm_pred$fit - (z_val * glm_pred$se.fit)
log_upr <- glm_pred$fit + (z_val * glm_pred$se.fit)
r_method_pred <- data.frame(
  prediction = exp(glm_pred$fit),
  lwr = exp(log_lwr),
  upr = exp(log_upr)
)

print(first_principle_pred)

```

```

      fit      X1      X1.1
1 153.6828 142.0139 166.3106

```

```
print(r_method_pred)
```

```

 prediction      lwr      upr
1   153.6828 142.0139 166.3106

```

They match!

2

```

set.seed(637)
true_beta0 = 1
true_beta1 = .5

```

```

var = 1
x <- 1:50
y1 <- rnorm(50, mean = true_beta0 + x*true_beta1, sd = sqrt(var))
y2 <- rnorm(50, mean = true_beta0, sd = sqrt(var))

```

(a) Fit intercept only model and model with covariate and find deviance

```

normal_deviance <- function(X, beta, y) {
  mu_hat <- X %*% beta
  sum((y - mu_hat)^2)
}

```

```

mod1_int <- glm(y1~1, family = gaussian)
mod1_full <- glm(y1~x, family = gaussian)
mod2_int <- glm(y2~1, family = gaussian)
mod2_full <- glm(y2~x, family = gaussian)

# Y1 Intercept Model
mod1_int_man_dev <- normal_deviance(model.matrix(y1~1), coef(mod1_int), y1)
mod1_int_r_dev <- mod1_int$deviance
# Y1 Full Model
mod1_full_man_dev <- normal_deviance(model.matrix(y1~x), coef(mod1_full), y1)
mod1_full_r_dev <- mod1_full$deviance
# Y2 Intercept Model
mod2_int_man_dev <- normal_deviance(model.matrix(y2~1), coef(mod2_int), y2)
mod2_int_r_dev <- mod2_int$deviance
# Y2 Full Model
mod2_full_man_dev <- normal_deviance(model.matrix(y2~x), coef(mod2_full), y2)
mod2_full_r_dev <- mod2_full$deviance

res <- data.frame(
  model = c(1, 1, 1, 1, 2, 2, 2, 2),
  fit = c('Intercept', 'Intercept', 'Full', 'Full', 'Intercept', 'Intercept', 'Full', 'Full'),
  method = c("manual", "R", "manual", "R", "manual", "R", "manual", "R"),
  value = c(mod1_int_man_dev, mod1_int_r_dev, mod1_full_man_dev, mod1_full_r_dev,
    mod2_int_man_dev, mod2_int_r_dev, mod2_full_man_dev, mod2_full_r_dev)
)
rownames(res) <- NULL
res

```

	model	fit method	value
1	1	Intercept manual	2520.86249
2	1	Intercept R	2520.86249
3	1	Full manual	56.96403
4	1	Full R	56.96403
5	2	Intercept manual	49.24842
6	2	Intercept R	49.24842
7	2	Full manual	49.03541
8	2	Full R	49.03541

(b) Describe how the models compare to the saturated models using Chi-squared

```
mod1_delta_dev <- mod1_int_man_dev-mod1_full_man_dev
mod1_p_val <- pchisq(mod1_delta_dev, 1, lower.tail = FALSE)

mod2_delta_dev <- mod2_int_man_dev-mod2_full_man_dev
mod2_p_val <- pchisq(mod2_delta_dev, 1, lower.tail = FALSE)
```

For the first dataset the difference in deviances between the Intercept and Full models is very large. We get a ΔD of 2463.898 which is an extremely large value. When we test this on the corresponding Chi-squared distribution we get a p-value of 0. Based on this we conclude that the saturated model is significantly better than the intercept model, and we conclude that our model should include the covariate. For the second dataset we get a ΔD of 0.213 which is small, and a p-value of 0.644, so we conclude that the saturated model is not significantly different. Thus, we do not include the covariate for the second dataset.

(c) Compare the models using delta-deviance. Is one model significantly better?

For this section I wasn't too sure what to do beyond what I already did in (b), so I decided to just include the ANOVA outputs and show they match what I did in (b), and yield the same conclusions. The ANOVA test uses the delta deviance so it feels like it answers the question? Really not sure what more to add beyond what's in (b)

```
anova(mod1_int, mod1_full)
```

Analysis of Deviance Table

```
Model 1: y1 ~ 1
Model 2: y1 ~ x
```

	Resid. Df	Resid. Dev	Df	Deviance	F	Pr(>F)
1	49	2520.86				
2	48	56.96	1	2463.9	2076.2	< 2.2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
anova(mod2_int, mod2_full)
```

Analysis of Deviance Table

Model 1: y2 ~ 1

Model 2: y2 ~ x

	Resid. Df	Resid. Dev	Df	Deviance	F	Pr(>F)
1	49	49.248				
2	48	49.035	1	0.21301	0.2085	0.65

Again, we keep the covariate in the first dataset, but would not include it in the second dataset.

3

```
b <- c(-2.4, .8)
cov_b <- matrix(c(.34, -.15, -.15, .10), byrow = TRUE, nrow = 2)
```

(a) For $x=2$, find $\hat{\eta}$ and $\hat{\mu}$

Because $\log(\mu_i) = \eta_i$, $\mu_i = e^{\eta_i}$

```
# X = intercept, actual value
x = t(c(1, 2))

eta_hat <- as.numeric(x%*%b)
mu_hat <- round(exp(eta_hat), 3)
```

We get an estimated $\hat{\eta}$ of -0.8, which gives an estimated $\hat{\mu}$ of 0.449.

(b) Find SE of $\hat{\eta}$ using $Cov(\beta)$

$$SE(\hat{\eta}) = \sqrt{x Cov(\beta) x^\top}$$

```
# Error of eta
se_eta <- sqrt(x %*% cov_b %*% t(x))
```

(c) Construct 95% interval on $\hat{\eta}$. Transform endpoints to get interval for $\hat{\mu}$

```
z_crit <- qnorm(0.975)
ci_eta_lower <- eta_hat - z_crit * se_eta
ci_eta_upper <- eta_hat + z_crit * se_eta

eta_hat_interval <- data.frame(
  lwr = round(ci_eta_lower, 3),
  upr = round(ci_eta_upper, 3)
)

mu_hat_interval <- data.frame(
  lwr = round(exp(ci_eta_lower), 3),
  upr = round(exp(ci_eta_upper), 3)
)
```

$\hat{\eta}$ Interval: -1.533, -0.067

$\hat{\mu}$ Interval: 0.216, 0.936

(d) Use delta method to approx. $SE(\hat{\mu})$. Use that to make 95% interval on $\hat{\mu}$

Delta Method: $\sqrt{n}(g(X_n) - g(\theta)) \xrightarrow{d} N(0, \sigma^2[g'(\theta)]^2)$

We have $SE(\hat{\eta}) \approx \sigma$

$$\Rightarrow \sqrt{n}(\exp(\mu) - \exp(\eta)) \xrightarrow{d} N(0, SE(\hat{\eta})^2 [\frac{1}{\eta}]^2)$$

So, we would say: $\mu \sim N(0, SE(\hat{\eta}) \frac{1}{\eta})^2$

```

se_mu <- se_eta / (eta_hat^2)
z_crit <- qnorm(0.975)
ci_mu_lower <- mu_hat - z_crit * se_mu
ci_mu_upper <- mu_hat + z_crit * se_mu
mu_hat_interval <- data.frame(
  lwr = round(exp(ci_mu_lower), 3),
  upr = round(exp(ci_mu_upper), 3)
)

```

New $\hat{\mu}$ Interval: 0.498, 4.928

(e) Compare the two intervals, explain why (c) is preferable

The intervals are different because the way they were obtained was different. The one in (c) first finds the interval on η , then transforms the end points. The second transforms η first, then finds endpoints on μ . This results in a much larger interval, whereas the method in (c) produces a much smaller interval, making it preferable.

4

(a) Show that $\frac{Z}{\sqrt{V/\nu}} \sim t_\nu$

$$Z \sim N(0, 1), V \sim \chi_\nu^2$$

For this, I am following the proof in Casella Berger page 223-224:

Start with the joint PDF:

$$f_{Z,V}(z, v) = \frac{1}{(2\pi)^{1/2}} e^{-z^2/2} \frac{1}{\Gamma(\nu/2)2^{\nu/2}} v^{(\nu/2)-1} e^{-v/2}$$

Use the transformation: $t = \frac{z}{\sqrt{v/\nu}}, w = v$

$$v = w, z = t\sqrt{\frac{w}{\nu}}, \frac{d}{dw}v = 1, \frac{d}{dw}z = \frac{t}{\sqrt{w\nu}}, \frac{d}{dt}v = 0, \frac{d}{dt}z = \sqrt{\frac{w}{\nu}}$$

This gives a Jacobian of $\sqrt{\frac{w}{\nu}}$

$$f_T(t) = \int_0^\infty f_{Z,V}(t\sqrt{\frac{w}{\nu}}, w) \sqrt{\frac{w}{\nu}} dw$$

$$f_T(t) = \frac{1}{(2\pi)^{-1/2}} \frac{1}{\Gamma(\nu/2)2^{\nu/2}} \int_0^\infty e^{-(1/2)t^2 w/\nu} w^{\nu/2-1} e^{-w/2} \sqrt{\frac{w}{\nu}} dw$$

$$f_T(t) = \frac{1}{(2\pi)^{-1/2}} \frac{1}{\Gamma(\nu/2)2^{\nu/2} \nu^{1/2}} \int_0^\infty e^{-(1/2)(1-t^2/\nu)w} w^{((\nu+1)/2)-1} dw$$

That's the kernel of a $Gamma(\frac{\nu+1}{2}, \frac{2}{(1+t^2/\nu)})$

$$f_T(t) = \frac{1}{(2\pi)^{1/2}} \frac{1}{\Gamma(\nu/2) 2^{\nu/2} \nu^{1/2}} \Gamma\left(\frac{\nu+1}{2}\right) \left[\frac{2}{1+t^2/\nu}\right]^{(\nu+1)/2}$$

That is the PDF for a t-distribution, showing that $\frac{Z}{\sqrt{V/\nu}} \sim t_\nu$

(b)

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y$$

Let c_j be the j th diagonal of $(X^\top X)^{-1}$. Show that $SE(\hat{\beta}_j) = s\sqrt{c_j}$

We know that $V(\hat{\beta}) = \sigma^2(X^\top X)^{-1}$

This gives the equivalent (in sampling) of $Var(\hat{\beta}) = s^2 c$

This is because the SE of a specific β only depends on the diagonals of the Variance, not the off-diagonals.

Taking the square root of each side: $SE(\hat{\beta}) = s\sqrt{c}$

If we want just the j th element of $SE(\hat{\beta})$, that only impacts c because s is a scalar, so we get:

$$SE(\hat{\beta}_j) = s\sqrt{c_j}$$

(c)

Show that $T_j = \frac{\hat{\beta}_j - \beta_j}{SE(\hat{\beta}_j)} \sim t_{n-k}$

So, we can rewrite $\frac{\hat{\beta}_j - \beta_j}{SE(\hat{\beta}_j)}$ as $\frac{\hat{\beta}_j - \beta_j}{s\sqrt{c_j}} = \frac{\hat{\beta}_j - \beta_j}{\frac{\sqrt{SSE * c_j}}{\sqrt{n-k}}}$

We know that SSE follows a Chi-squared distribution, and $\hat{\beta}_j - \beta_j$ follows a 0-centered normal, meaning we can just use the same identity we proved in (a) to say that this is a transformation of a normal and chi-squared:

With $\hat{\beta}_j - \beta_j$ as Z , $SSE * c_j$ as V , and $n - k$ as ν , we get: $\frac{Z}{\sqrt{V/\nu}} \sim t_\nu$

Plugging everything back in we get:

$$\frac{\hat{\beta}_j - \beta_j}{\frac{\sqrt{SSE * c_j}}{\sqrt{n-k}}} \sim t_{n-k}$$

(d)

The uncentered statistic $\frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$ has the t_{n-k} distribution only when the null hypothesis is that $\beta_j = 0$ because if $\beta_j \neq 0$, we no longer have a 0-centered normal in the numerator, meaning that we can no longer use the identity from (a) to say that it's a t-distribution.

(e)

$a \in \mathbb{R}^k$. Derive the exact distribution of:

$$\frac{a^\top \hat{\beta} - a^\top \beta}{s \sqrt{a^\top (X^\top X)^{-1} a}}$$

First we recognize that this is basically just $a^\top * [\frac{\hat{\beta} - \beta}{s \sqrt{X^\top X}^{-1}}]$

So, this is really just a transformation where $g(\theta) = a^\top \theta$. We can then use the Delta Method in large n, where:

$$\frac{1}{\sqrt{n}}(\hat{\beta} - \beta) \sim N(0, s^2)$$

So now $\frac{1}{\sqrt{n}}(g(\hat{\beta}) - g(\beta)) \sim N(0, s^2[g'(\beta)]^2)$

$$g(\beta) = a^\top \beta, g'(\beta) = a^\top$$

$$\implies \frac{1}{\sqrt{n}}(a^\top \hat{\beta} - a^\top \beta) \sim N(0, s^2 a^\top a)$$

(f)

The normal linear model gives an exact t-statistic because it's a direct closed-form solution (an exact transformation). However, other GLMs only give an approximate normal because they are reliant on the Central Limit Theorem which says they are normal asymptotically (ie. they are never perfectly normal)